QUESTIONS

5/5 answered

Q1 ATM Withdrawal – Exception Handling Debugging

10 marks

An ATM application is used to process cash withdrawals. The following Java program contains programming errors.

Task:

1. Run the given program and identify the errors.
2. Debug and modify the program.
3. Submit the COMPLETE corrected Java program.
4. The corrected program must handle insufficient balance, zero withdrawal, invalid input, and unexpected errors.
5. Use appropriate try-catch-finally exception handling.
6. The corrected program must execute successfully for all valid test cases.

Faulty Program:

```
import java.util.Scanner;

class ATM {
public static void main(String[] args) {

Scanner sc = new Scanner(System.in);

System.out.print("Enter account balance: ");
int balance = sc.nextInt();

System.out.print("Enter withdrawal amount: ");
int amount = sc.nextInt();

try {
int remaining = balance / amount;

if (amount > balance) {
System.out.println("Insufficient balance");
} else {
System.out.println("Withdrawal successful");
```

Your Fixed Code

```java
import java.util.*;
public class Main{
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        try{
            int balance=sc.nextInt();
            int amount=sc.nextInt();
            if(amount==0){
                throw new ArithmeticException("Withdrawal amount cannot
            }
            if(amount<0){
                System.out.println("Invalid withdrawal amount");
            }
            else if(amount>balance){
                System.out.println("Insufficient balance");
            }
            else{
                int remainingBalance=balance-amount;
                System.out.println("Withdrawal successful");
                System.out.println("Remaining balance = "+remainingBala
            }
        }
        catch(InputMismatchException e){
            System.out.println("Invalid input");
        }
        catch(ArithmeticException e){
            System.out.println(e.getMessage());
        }
        catch(Exception e){
            System.out.println("Transaction failed");
        }
        finally{
            System.out.println("Transaction completed");
```

Test Input

```
10000
3000
```

Output

```
Withdrawal successful
Remaining balance = 7000
Transaction completed
```

Pending manual review

CO3-AT1-Debugging and Optimization

← Back

QUESTIONS

5/5 answered

Q2 Hospital Registration – NullPointerException Debugging

10 marks

A hospital patient registration application crashes when patient information is unavailable.

Debug the following complete Java program.

Task:

1. Identify the runtime error.
2. Modify the program to handle a missing patient name.
3. Handle invalid age input.
4. Use try-catch-finally appropriately.
5. The program must always display "Registration completed".
6. Submit the COMPLETE corrected Java program.

Faulty Program:

```
import java.util.Scanner;

class PatientRegistration {

public static void main(String[] args) {

Scanner sc = new Scanner(System.in);

System.out.print("Enter patient name: ");
String inputName = sc.nextLine();

String name;

if (inputName.equals("NULL")) {
name = null;
} else {
```

SIMATS ENGINEERING

Secure Exit

SIMATS Online Programming Lab

JAVA PROGRAMMING

K.Dhilip
192365013RD

Your Fixed Code

```java
import java.util.*;
public class Main{
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        try{
            String inputName=sc.nextLine();
            String name;
            if(inputName.equals("NULL")){
                name=null;
            }else{
                name=inputName;
            }
            int age=sc.nextInt();
            if(name==null){
                System.out.println("Patient name is unavailable");
            }else{
                System.out.println("Patient Name: "+name.toUpperCase());
            }
            if(age>=18){
                System.out.println("Adult patient");
            }else{
                System.out.println("Minor patient");
            }
        }catch(InputMismatchException e){
            System.out.println("Invalid age input");
        }
        catch(Exception e){
            System.out.println("Registration failed");
        }
        finally{
            System.out.println("Registration completed");
        }
    }
}
```

Test Input

```
Arun
25
```

Output

```
Patient Name: ARUN
Adult patient
Registration completed
```

Pending manual review

CO3-AT1-Debugging and Optimization

← Back

QUESTIONS

5/5 answered

Q3 E-Commerce Product Search – Array Exception Debugging

10 marks

An online shopping application allows customers to select a product using a product number from 1 to 5.

The following program contains an array indexing error.

Task:

1. Identify the error.
2. Correct the product-number and array-index logic.
3. Handle invalid product numbers.
4. Handle non-numeric input.
5. Submit the COMPLETE corrected Java program.
6. The program must display "Search completed" after processing.

Faulty Program:

```
import java.util.Scanner;

class ProductSearch {

public static void main(String[] args) {

Scanner sc = new Scanner(System.in);

String[] products = {
"Laptop",
"Mobile",
"Tablet",
"Headphone",
"Keyboard"
};
```

SIMATS ENGINEERING

Secure Exit

SIMATS Online Programming Lab

JAVA PROGRAMMING

K.Dhilip
192365013RD

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 import java.util.*;
2 public class Main{
3     public static void main(String[] args){
4         Scanner sc=new Scanner(System.in);
5         String[] products={"Laptop","Mobile","Tablet","Headphone","Keyb
6         try{
7             int productNumber=sc.nextInt();
8             if(productNumber<1||productNumber>5){
9                 System.out.println("Invalid product number");
10            }else{
11                System.out.println("Selected product: "+products[produc
12            }
13        }
14        catch(InputMismatchException e){
15            System.out.println("Invalid input");
16        }
17        catch(Exception e){
18            System.out.println("Search failed");
19        }
20        finally{
21            System.out.println("Search completed");
22        }
23    }
24 }
```

Submitted

Test Input

```
5
```

Output

```
Selected product: Keyboard
Search completed
```

Pending manual review

CO3-AT1-Debugging and Optimization

← Back

QUESTIONS

5/5 answered

Q4 Railway Reservation – Synchronization Debugging

10 marks

A railway reservation system has only one available seat. Two customers attempt to reserve the seat simultaneously.

The following complete Java program contains a synchronization problem.

Task:

1. Identify the race condition.
2. Debug the program using the synchronized keyword.
3. Ensure that only ONE customer can reserve the available seat.
4. The second customer must receive "No seat available".
5. Submit the COMPLETE corrected Java program.

Faulty Program:

```
class Reservation {

int seats = 1;

void bookSeat(String customer) {

if (seats > 0) {

System.out.println(customer
+ " is booking a seat");

try {
Thread.sleep(100);
}
catch (InterruptedException e) {
System.out.println("Thread interrupted");
```

```java
import java.util.*;
public class Main{
    public static void main(String[] args){
        Reservation reservation=new Reservation();
        Customer c1=new Customer(reservation,"Customer 1");
        Customer c2=new Customer(reservation,"Customer 2");
        c1.start();
        c2.start();
    }
}
class Reservation{
    int seats=1;
    synchronized void bookSeat(String customer){
        if(seats>0){
            System.out.println(customer+" is booking a seat");
            try{
                Thread.sleep(100);
            }catch(InterruptedException e){
                System.out.println("Thread interrupted");
            }
            seats--;
            System.out.println(customer+" booking successful");
        }else{
            System.out.println(customer+" - No seat available");
        }
    }
}
class Customer extends Thread{
    Reservation reservation;
    String customerName;
    Customer(Reservation reservation,String customerName){
        this.reservation=reservation;
        this.customerName=customerName;
    }
    public void run(){
```

stdin input (optional)

Output

```
Customer 1 is booking a seat
Customer 1 booking successful
Customer 2 - No seat available
```

Pending manual review

CO3-AT1-Debugging and Optimization

← Back

QUESTIONS

5/5 answered

Q5 Bank Account – Race Condition Debugging

10 marks

A bank account contains ₹10,000. Two ATM threads simultaneously attempt to withdraw ₹7,000 and ₹6,000.

The following program contains a race condition.

Task:

1. Identify the race condition.
2. Identify the critical section.
3. Explain why both threads may pass the balance check.
4. Modify the complete program using synchronized.
5. Ensure that the balance can never become negative.
6. Submit the COMPLETE corrected Java program.

Faulty Program:

```
class BankAccount {

int balance = 10000;

void withdraw(int amount) {

if (balance >= amount) {

System.out.println(
Thread.currentThread().getName()
+ " withdrawing " + amount);

try {
Thread.sleep(100);
}
```

Your Fixed Code

```java
public class Main{
    public static void main(String[] args)throws InterruptedException{
        BankAccount account=new BankAccount();
        ATM atm1=new ATM(account,7000);
        ATM atm2=new ATM(account,6000);
        atm1.setName("ATM-1");
        atm2.setName("ATM-2");
        atm1.start();
        atm2.start();
        atm1.join();
        atm2.join();
        System.out.println("Final Balance = "+account.balance);
    }
}
class BankAccount{
    int balance=10000;
    synchronized void withdraw(int amount){
        if(balance>=amount){
            System.out.println(Thread.currentThread().getName()+" withc
            try{
                Thread.sleep(100);
            }catch(InterruptedException e){
                System.out.println("Interrupted");
            }
            balance=balance-amount;
            System.out.println(Thread.currentThread().getName()+" withc
        }else{
            System.out.println(Thread.currentThread().getName()+" insu
        }
    }
}
class ATM extends Thread{
    BankAccount account;
    int amount;
```

Test Input

stdin input (optional)

Output

```
ATM-1 withdrawing 7000
ATM-1 withdrawal successful
ATM-2 insufficient balance
Final Balance = 3000
```

Pending manual review